REFERENCE GUIDE · BOTH QUEUES

The Eval Sandbox

A QC pass — and two writers — that you run on your own task before you submit it. Six slash commands, typed into a chat panel. Nothing to install, no task ID, no files to open. **Four belong to Honeybee SxS and apply to both of its task types; `/item` and `/rubric` belong to Prompt and Rubrics and are Type 2 (Aspirational) only.**

CONTENTS

1. What it is
2. The two kinds of command
3. Snapshot, then Save
4. The six commands
5. Reading the result
6. How to use it, step by step
7. When each one is due
8. Why we do this
9. Questions people have asked

1. What it is

Most tasks carry an **eval sandbox** step: a small IDE, like VS Code, with Claude Code already running inside it and the eval skills already installed. You use it to check your own work at several points before submitting.

WHICH COMMANDS ARE YOURS — CHECK THIS FIRST

The six commands split across the two queues, and one split matters more than the other:

- **Honeybee SxS** — `/trajectory`, `/grading`, `/dimensions`, `/sxs`. All four apply to **both SxS task types**, Traffic and Aspirational. None of them reads a universe — they read your trajectory, your grades and your ratings — so the task type makes no difference here.
- **Honeybee Prompt and Rubrics** — `/item` and `/rubric`, and these are **Type 2 (Aspirational) only**.

A Type 1 (Traffic) task in Prompt and Rubrics has no eval sandbox step at all. That is a real, permanent difference: `/item` checks entities against an assigned universe, and a Traffic task has none. If you are on a Traffic task in that queue and cannot find the sandbox, that is the expected state and not something to raise — **this whole guide simply does not apply to you.**

STATUS ON HONEYBEE SXS: MISSING FROM MOST TASKS, AND MAY NOT RETURN

Separately from the split above, **most SxS tasks currently in the queue have no eval sandbox steps**, and the current direction is to move this QC coverage to **in-task endpoint evals** — automated checks that run against the trajectory the task now captures for each model, rather than commands a contributor types. **The four SxS commands may not be re-added.**

This guide stays published either way, so the workflow is documented if the commands come back and so the standards each command enforces stay readable — the things `/trajectory` and `/sxs` look for are the same things a reviewer looks for, whether or not you can run them yourself.

What to do: if the commands are on your task, run them; if they are not, complete and submit the task without them. Do not report their absence, do not wait for them, do not skip the task. Every command here is a self-check on work you have already produced, so nothing in your submission depends on one having run. **This is not related to the SxS task type** — none of the four reads a universe.

NOT TO BE CONFUSED WITH THE TRAJECTORY CAPTURE SANDBOX

Honeybee SxS tasks now carry a **second, different sandbox** after each model's turn collection: you paste that model's share link, the conversation is exported, and you press **Capture Files** then **Save** so it renders on the following **Agent Execution** step. **That step is always required and takes no slash commands** — and it is what the in-task evals read. It is documented in **Honeybee SxS — Contributor Guidelines, section 10**, not here.

IT IS JUST CHATTING WITH CLAUDE

Wait for the green **Sandbox files ready** message at the bottom. Then click into the **Claude Code** panel on the right, type the slash command into the message box, and press Enter.

Ignore the rest of the screen. You never need to run anything yourself, open a file, create a folder, or type a task ID. One sandbox is one task.

If the step gave you a **copy-paste box**, paste that whole block as your first message. If it did not, the command asks you for one thing at a time and confirms what arrived — a count or a length — so a bad paste is obvious straight away.

You do not have to remember any of this

Just do your task as normal. **When you reach a step whose label or instruction tells you to run an eval, that is your cue** — scroll back up to the sandbox and type the command it names. If you have not hit an instruction telling you to run one, there is nothing to do; keep going.

The sandbox sits at the early part of the flow, right after the GTFA, so for the later checks you will be scrolling back up to it.

2. The two kinds of command

Six commands, in two kinds. **Do not mix them up** — they behave differently and the difference matters.

Kind	Commands	What it does
Eval commands Blocking	`/item` · `/trajectory a|b|c` `/dimensions a|b|c` · `/sxs ab|ac`	**Check what you wrote.** A FAIL or UNAUDITABLE means you fix it, re-run, then Snapshot and Save before you move on.
Prefilling commands Not blocking	`/rubric` · `/grading a|b|c`	**Write the rubric and the met/not-met grades for you.** The form does not auto-prefill when the command finishes — Snapshot then Save is what prefills it.

The letter on the end

Three of the commands need to know which product you mean, so you add a letter after a space: `a` = Product A, `b` = Product B, `c` = Product C. So you would type `/grading b`.

For the side-by-side you name ***the pair*** you compared instead: `/sxs ab` for A vs B, `/sxs ac` for A vs C.

> **WHICH LETTER IS WHICH PRODUCT**
>
> The letters map to the product slots as your task labels them — **read the labels, do not assume from a previous task.** On a task with a base model and one comparison model you will normally use `a` and `b` only. Running a command with the wrong letter checks the wrong run and the finding will look nonsensical, which is usually how people notice.

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-10-14T07:19:20.534Z

3. Snapshot, then Save

> **EVERY COMMAND, EVERY TIME**
>
> **After every command — both kinds — press Snapshot, then Save.** Until you do, the result exists only inside the sandbox and will not show up on the task. Skipping this looks exactly like the command never ran.
>
> It matters twice as much for `/rubric` and `/grading`: **those two do not auto-prefill when the command finishes.** Snapshot then Save is what fills the rubric or the met/not-met grades into the form. If the command completed and the form is still empty, that is expected — you have not snapshotted yet.

And never copy the chat output into the form by hand. Snapshot and Save is the mechanism; hand-copying introduces transcription errors and defeats the point of the writers.

4. The six commands

The list below is just so you know what each command is looking for when its turn comes.

`/item` BLOCKING PROMPT AND RUBRICS · TYPE 2 ONLY

Checks your task itself — the prompt, the target outcome and the GTFA.

- **Phantom entities** — names, files or figures cited in your prompt or GTFA that are not actually anywhere in the universe. **This check is why the command is Aspirational-only:** it grades against the assigned universe, and a Traffic task has none.
- **Front-loading** — a prompt that asks for everything at once
- **Ungradable outcome elements** — "looks good", "remains readable"
- **Contradictions** — a GTFA that disagrees with the prompt or the target outcome

---

Input: opening prompt, target outcome, GTFA

`/rubric` NOT BLOCKING PROMPT AND RUBRICS · TYPE 2 ONLY

Writes your weighted rubric from the target outcome and GTFA, then self-checks it so it will not ship dead weight. **Do not paste a rubric into it.**

- **Dead weight** — it will not include criteria no model could pass

- Every target-outcome element gets at least one criterion; packed lines are split here
- Weights 1–3 and Outcome-quality categories applied
- Wording two graders would score the same way

Input: opening prompt, target outcome, GTFA — already on disk after `/item`, or paste the copy box

THE SANDBOX RUBRIC AND THE TASK RUBRIC ARE NOT THE SAME OBJECT

Both get called "the rubric," which is why this trips people up. **The rubric `/rubric` produces lives in the sandbox and carries extra annotations the eval needs** — most visibly an `[Inclusive]` or `[Exclusive]` label on each criterion, which controls how that criterion is counted during evaluation. The rubric you author and submit on the task itself has no field for those labels.

You never type these labels by hand. The sandbox applies them when it writes the rubric. Do not add them to criterion titles in the task form, and do not treat their absence there as a mistake.

`/trajectory a` BLOCKING PER PRODUCT SXS QUEUE

Checks your captured run. Type `/trajectory a` for Product A, `/trajectory b` for Product B, and so on.

- Wrong turn count, gaps, missing turns
- Missing or duplicated share links
- **Padding turns** — turns that exist only to reach the count
- Hidden information revealed too early, or all at once

Input: turn-by-turn links + final trajectory link (or the `trajectory.json`)

`/grading a` NOT BLOCKING SXS QUEUE

Writes met/not-met for that run, with turn anchors. **Do not paste a grading sheet**, and run one product per command — never B and C together.

- Every rubric criterion gets a 0 or 1, with a real turn as the anchor
- Dead-weight and unreachable criteria are tagged **excluded**, not silent zeros
- Raw score and adjusted score both reported
- Arithmetic that adds up

Input: the trajectory export for that product, plus the rubric `/rubric` already wrote

`/dimensions a` BLOCKING PER PRODUCT SXS QUEUE

Checks your 8 dimension ratings for that product.

- Wrong turns cited as relevant
- Generic filler justifications
- Ratings that do not match the issues you listed

Input: your 8 ratings + justifications + selected turns

`/sxs ab` BLOCKING PER PAIR SXS QUEUE

Checks your side-by-side comparison. Type `/sxs ab` for A vs B, `/sxs ac` for A vs C.

- **Direction flip** — picking one product while scoring the other higher
- **Understated strength** — a big score gap described as "slight"
- Claims not grounded in either run
- **Rubric vs. dimension contradiction** — penalising something in the rubric, then praising it in the dimensions

Input: your selection, strength, key differentiators, and summary

5. Reading the result

Every command returns one of four scores, plus the specific thing to fix.

5 PASS

Clean. Move on.

3 NON-FAIL

Works, but has real weaknesses worth fixing. On a blocking command this is a judgement call, not a stop.

2 **FAIL**

Would be rejected. On a blocking command, fix before you move on.

1 **UNAUDITABLE**

Something needed to check it is missing. Treated the same as a fail.

> **IT GRADES TO THE LOWEST SCORE, NOT THE AVERAGE**
>
> One dimension at 2 makes the whole thing FAIL. That is deliberate — it is how the work gets reviewed downstream, so there is no point averaging a blocker away.

Findings point at where to fix in your own terms: `GTFA ¶1`, `Outcome 23`, `#7 anchor`, `Turn 14 link`.

6. How to use it, step by step

1

Wait for a step to tell you to run an eval

You will see it in the step label or instruction. Until then, there is nothing to do.

2

Scroll back up to the eval sandbox step and open it

Wait for the green **Sandbox files ready** at the bottom before you do anything.

3

Click into the chat panel on the right

The one headed **Claude Code**. Type into the message box at the bottom of it.

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-10-14T07:19:20.534Z

4

Type the command and press Enter

For example `/item`. No task ID — just the command, plus a product letter on the three that need one.

5

Answer its questions in chat

If the step has a copy-paste box, paste that whole block. Otherwise it asks for one thing at a time and confirms what arrived, so a bad paste is obvious immediately. **Never paste a rubric into `/rubric` or a grading sheet into `/grading`.**

6

Read the verdict and fix what it names

On a blocking command — `/item`, `/trajectory`, `/dimensions`, `/sxs` — a FAIL or UNAUDITABLE means you do not move on yet.

7

Press Snapshot, then Save

Every command, every time. That is what makes the result show up on the task.

8

Re-run it if you had to fix something

Your answers in chat are still there, so a re-run is quick. Then Snapshot and Save again, and move to the next command.

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-10-14T07:19:20.534Z

7. When each one is due

The step labels tell you, so you do not have to track this. For reference they come up in this order — and **each one reads what the previous one stored**, so following the labels is also the fastest route.

Run after you finish…	Command	Queue	Gate
GTFA	`/item`	Prompt and Rubrics · **Type 2 only**	**Blocking**
Item cleared	`/rubric` ***writes the rubric***	Prompt and Rubrics · **Type 2 only**	Not blocking
Capturing a run	`/trajectory a b c`	SxS	**Blocking**
Run captured	`/grading a b c` ***writes met/not met***	SxS	Not blocking
Its 8 dimension ratings	`/dimensions a b c`	SxS	**Blocking**
Each side-by-side	`/sxs ab ac`	SxS	**Blocking**

Every row still needs Snapshot then Save when the command finishes — the two writers included. You can run the commands out of order if you need to, but later ones read what earlier ones stored, so in order is faster.

8. Why we do this

These are the failures the suite is calibrated on. All of them got flagged downstream, and all of them are cheap to catch while you are still holding the task:

- **Dead weight** — tasks rejected with 31% and 59% of rubric weight unpassable
- **Phantoms** — a GTFA citing a file, person or figure that is not in the universe
- **Impossible anchors** — grading cites a turn that was never in the run
- **False zeros** — criteria marked failed that the run actually passed
- **Rubric/dimension contradiction** — the same thing penalised, then praised
- **Understated SxS strength** — a large gap called "slight"

> **WHY A SECOND PASS BY YOU WOULD NOT FIND THESE**
>
> None of them are findable by re-reading your own work — that is the point. **A second pass by the same eyes tends to confirm what it already believes.**

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-10-14T07:19:20.534Z

9. Questions people have asked

I don't see the eval step on my task.

Check what queue and task type you are on first. On a **Type 1 (Traffic) task in Prompt and Rubrics** there is no eval step by design and nothing to raise. On a **Honeybee SxS task** the eval commands are missing from most tasks and may not be re-added, with the coverage moving to in-task endpoint evals — **complete and submit without them** rather than skipping the task or waiting. If the sandbox you are looking at asks for a share link instead of offering a Claude Code panel, that is the **trajectory capture** step, which is a different thing and is always required.

Do I have to fix everything it finds?

On a blocking command — `/item`, `/trajectory`, `/dimensions`, `/sxs` — fix anything scored 2 (FAIL) or 1 (UNAUDITABLE) before you move on. A 3 (NON-FAIL) is a judgement call. `/rubric` and `/grading` are not blocking: Snapshot and Save so the form prefills, then continue.

Do I really have to Snapshot and Save after every command?

Yes. Every command, including the two that write files. Until you Snapshot and Save, the result is only inside the sandbox and will not show up on the task.

`/rubric` or `/grading` finished, but the form is still empty.

That is expected. Those two do not auto-prefill when the command finishes. Press Snapshot, then Save — that is when the rubric or the met/not-met grades prefill into the form.

It says a name in my GTFA is a phantom, but I'm sure I saw it.

Tell it in chat — say which document you saw it in, and it will re-read that document in full and come back to you. It flags when it has only skimmed a very large file, so it will not quietly call something a phantom on a partial read. **If it is genuinely absent, it is a phantom even if it feels right.**

It flagged a filename that doesn't exist yet.

Absent is not automatically a phantom. If the string names something **the model is supposed to create** — a new file, a new folder, a draft subject — its absence is expected and is not a defect. It spells that split out for you under the results.

Can I run the commands out of order?

Yes, but later ones read what earlier ones stored. `/rubric` wants `/item` first; `/grading` wants the rubric and that product's trajectory. In order is faster.

Do I paste my rubric or my grades?

No. `/rubric` writes the rubric and `/grading a` (then `b`, then `c`) writes met/not met. Snapshot then Save prefills the form — do not copy the chat into the form by hand. If the step shows a copy-paste box, that dump is ***the inputs***, not a rubric or a grading sheet.

Do I need a task ID or a task folder name?

No. One sandbox is one task. Do not type a folder name, and do not create one.

My target outcome says an image "stays readable" — how does it check that?

It extracts the image and looks at it, and measures the number where a claim is quantitative. **A visual claim nobody looked at counts as unverified, not passed.**

The form asks for one ranking, but I compared A vs B and A vs C.

Score `/sxs ab` and `/sxs ac` separately — these are pairwise comparisons, not a three-way rank. If a form still has a single ranking slot, mark it N/A and **do not invent a 1st/2nd/3rd across all products.**

Do the `[Inclusive]` / `[Exclusive]` labels apply to the rubric I wrote in Queue 1?

No. See the section below — this catches people out constantly, because both things are called "the rubric."

Questions, feedback or blockers — send them to the War Room.

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-10-14T07:19:20.534Z